



LABORATORIS

PROGRAMACIÓ CIENTÍFICA

Sessió 1: El nostre primer programa en C

EL NOSTRE PRIMER PROGRAMA

Com es dissenya un programa?

1 **Identificar** quin és el problema que hem de resoldre amb el nostre programa

E.g. "Calcular l'àrea d'un rectangle"

Com es dissenya un programa?

1 **Identificar** quin és el problema que hem de resoldre amb el nostre programa

E.g. "Calcular l'àrea d'un rectangle"

2 **Analitzar** les necessitats que ha de cobrir l'aplicació, és a dir, les entrades i les sortides

E.g. Entrada: dimensions (base i altura) del rectangle en centímetres

*Sortida: L'àrea del rectangle en centímetres quadrats calculat com: $\text{Àrea} = \text{base} * \text{altura}$*

Com es dissenya un programa?

1 **Identificar** quin és el problema que hem de resoldre amb el nostre programa

E.g. "Calcular l'àrea d'un rectangle"

2 **Analitzar** les necessitats que ha de cobrir l'aplicació, és a dir, les entrades i les sortides

E.g. Entrada: dimensions (base i altura) del rectangle en centímetres

*Sortida: L'àrea del rectangle en centímetres quadrats calculat com: $\text{Àrea} = \text{base} * \text{altura}$*

3 **Dissenyar** l'algorisme que soluciona aquest problema i el joc de proves

Com es dissenya un programa?

- 1

Identificar

quin és el problema que hem de resoldre amb el nostre programa

E.g. "Calcular l'àrea d'un rectangle"
- 2

Analitzar

les necessitats que ha de cobrir l'aplicació, és a dir, les entrades i les sortides

E.g. Entrada: dimensions (base i altura) del rectangle en centímetres

Sortida: L'àrea del rectangle en centímetres quadrats calculat com: $\text{Àrea} = \text{base} * \text{altura}$
- 3

Dissenyar

l'algorisme que soluciona aquest problema i el joc de proves

```
algorisme area_rectangle és
var
    base_cm, altura_cm: enter;
    area_cm2 : enter;
fvar
inici
    base_cm := 5;
    altura_cm := 2;
    area_cm2 := base * altura;
    escriure("L'area és", area_cm2);
falgorisme
```

Prova	Entrades		Sortides
	base_cm	altura_cm	àrea_cm2
Prova de fum	3	2	6
Altre prova	5	10	50
Números grans	123	456	56088
Números petits	2	2	4

El Joc de Proves és **essencial**: conté les proves que li farem al nostre programa per tal d'estar segurs que funcionarà per qualsevol entrada vàlida. Per exemple, l'algorisme anterior funcionaria si `base_cm = 4,5cm`?

Com es dissenya un programa?

4 **Implementar** el programa en el llenguatge de programació que haguem triat

```
1 /** Programa que calcula l'àrea d'un rectangle a partir de la base i l'altura.
2 */
3 #include <stdio.h>
4 int main(){
5
6     int base_cm, altura_cm;
7     int area_cm2;
8
9     base_cm = 5;
10    altura_cm = 2;
11
12    // Càlcul de l'àrea
13    area_cm2 = base_cm * altura_cm;
14
15    printf("L'area del rectangle és %d\n", area_cm2);
16    return 0;
17 }
```


Com es dissenya un programa?

4 Implementar el programa en el llenguatge de programació que haguem triat

```
1 /** Programa que calcula l'àrea d'un rectangle a partir de la base i l'altura.
2 */
3 #include <stdio.h>
4 int main(){
5
6     int base_cm, altura_cm;
7     int area_cm2;
8
9     base_cm = 5;
10    altura_cm = 2;
11
12    // Càlcul de l'àrea
13    area_cm2 = base_cm * altura_cm;
14
15    printf("L'area del rectangle és %d\n", area_cm2);
16    return 0;
17 }
```

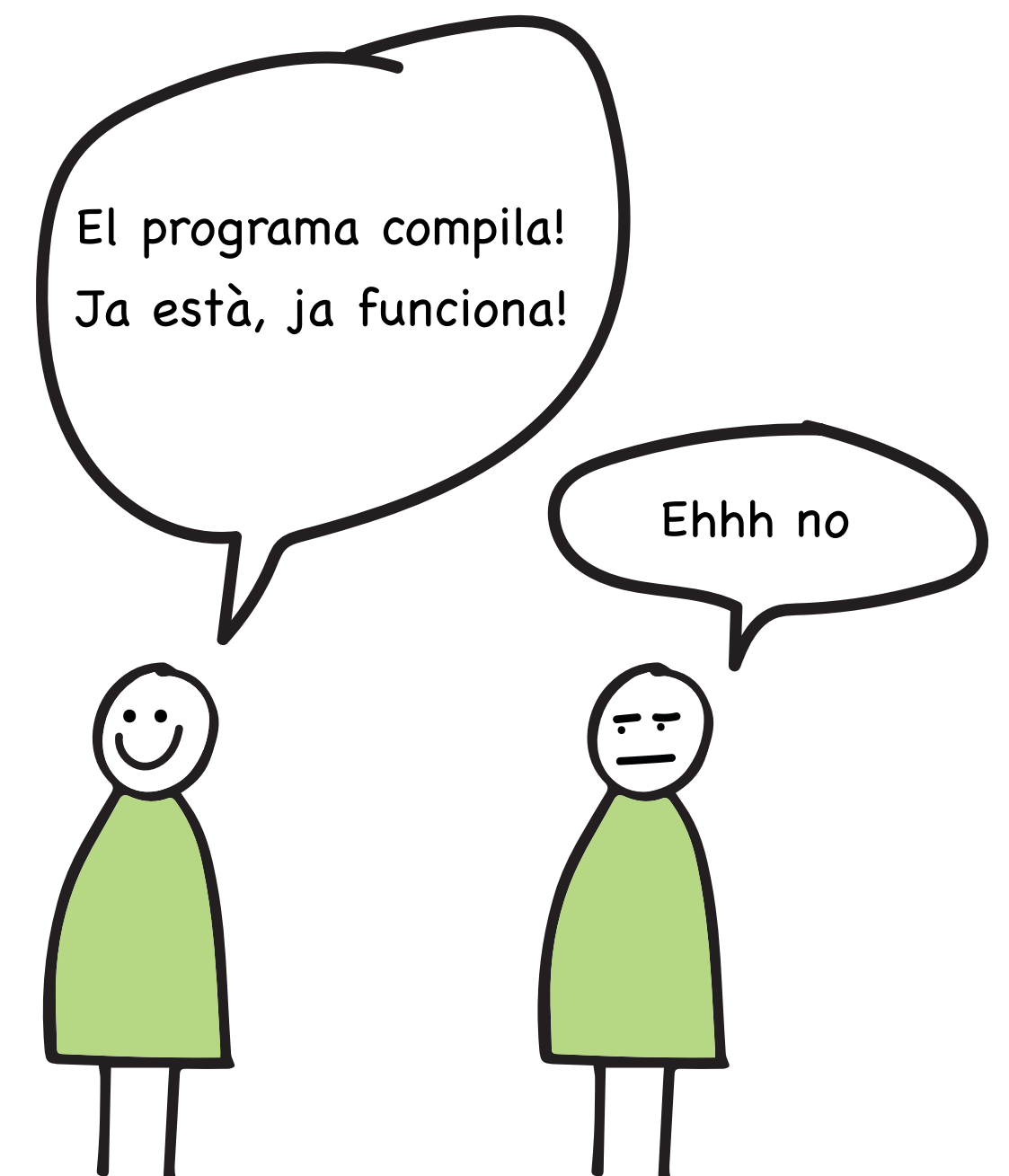
5 Executar i comprovar que el programa funciona segons el que esperàvem segons el joc de proves dissenyat.

Com es dissenya un programa?

4 Implementar el programa en el llenguatge de programació que haguem triat

```
1 /** Programa que calcula l'àrea d'un rectangle a partir de la base i l'altura.
2 */
3 #include <stdio.h>
4 int main(){
5
6     int base_cm, altura_cm;
7     int area_cm2;
8
9     base_cm = 5;
10    altura_cm = 2;
11
12    // Càlcul de l'àrea
13    area_cm2 = base_cm * altura_cm;
14
15    printf("L'area del rectangle és %d\n", area_cm2);
16    return 0;
17 }
```

5 Executar i comprovar que el programa funciona segons el que esperàvem segons el joc de proves dissenyat.



El nostre primer programa

- Avui aprendrem a "parlar" en llenguatge C.
- La única manera d'aprendre un nou llenguatge de programació és escrivint programes. El primer programa per excel·lència en tots els llenguatges és el Hello World.

El nostre primer programa

- Avui aprendrem a "parlar" en llenguatge C.
- La única manera d'aprendre un nou llenguatge de programació és escrivint programes. El primer programa per excel·lència en tots els llenguatges és el Hello World.

```
algorisme hello_world és  
inici  
    $ Escrivim un missatge per  
    pantalla  
    escriure ("Hello, World!");  
falgorisme
```


El nostre primer programa

- Avui aprendrem a "parlar" en llenguatge C.
- La única manera d'aprendre un nou llenguatge de programació és escrivint programes. El primer programa per excel·lència en tots els llenguatges és el Hello World.

```
algorisme hello_world és
inici
    $ Escrivim un missatge per
    pantalla
    escriure ("Hello, World!");
falgorisme
```



```
1 #include <stdio.h>
2
3 int main(){
4     /* Escrivim un missatge per pantalla */
5     printf("Hello, World!\n");
6     return 0;
7 }
```

El nostre primer programa: Hello World!

- Escriviu aquest programa i guardeu-lo on ho sapigueu localitzar (e.g. al Desktop) amb el nom **hello.c**

El nostre primer programa

- Estructura d'aquest codi senzill



```
1 #include <stdio.h>
2
3 int main(){
4     /* Escrivim un missatge per pantalla */
5     printf("Hello, World!\n");
6     return 0;
7 }
```

El nostre primer programa

- Estructura d'aquest codi senzill

Li diu al compilador que ens permeti fer servir les funcions incloses a la llibreria "**stdio.h**". Sense aquesta línia no es podria utilitzar el procediment **printf**, el compilador donaria un error.



```
1 #include <stdio.h>
2
3 int main(){
4     /* Escrivim un missatge per pantalla */
5     printf("Hello, World!\n");
6     return 0;
7 }
```


El nostre primer programa

- Estructura d'aquest codi senzill

El programa es posa dins el procediment principal del programa (**main**). En un programa només hi ha un main. Les funcions que creem nosaltres podran tenir el nom que vulguem, però el main s'ha de dir main. El programa es comença a executar pel main.

Li diu al compilador que ens permeti fer servir les funcions incloses a la llibreria "**stdio.h**". Sense aquesta línia no es podria utilitzar el procediment **printf**, el compilador donaria un error.

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. It contains C code with line numbers 1 through 7. An arrow points from the text 'Li diu al compilador...' to line 1, and another arrow points from the text 'El programa es posa dins el...' to line 3.

```
1 #include <stdio.h>
2
3 int main(){
4     /* Escrivim un missatge per pantalla */
5     printf("Hello, World!\n");
6     return 0;
7 }
```


El nostre primer programa

- Estructura d'aquest codi senzill

El programa es posa dins el procediment principal del programa (**main**). En un programa només hi ha un main. Les funcions que creem nosaltres podran tenir el nom que vulguem, però el main s'ha de dir main. El programa es comença a executar pel main.

Li diu al compilador que ens permeti fer servir les funcions incloses a la llibreria "**stdio.h**". Sense aquesta línia no es podria utilitzar el procediment **printf**, el compilador donaria un error.

```
1 #include <stdio.h>
2
3 int main(){
4     /* Escrivim un missatge per pantalla */
5     printf("Hello, World!\n");
6     return 0;
7 }
```

Aquesta línia no és codi, és un comentari. Tot el que estigui entre els símbols `/* */` no s'executarà

El nostre primer programa

- Estructura d'aquest codi senzill

El programa es posa dins el procediment principal del programa (**main**). En un programa només hi ha un main. Les funcions que creem nosaltres podran tenir el nom que vulguem, però el main s'ha de dir main. El programa es comença a executar pel main.

Li diu al compilador que ens permeti fer servir les funcions incloses a la llibreria "**stdio.h**". Sense aquesta línia no es podria utilitzar el procediment **printf**, el compilador donaria un error.

```
1 #include <stdio.h>
```

```
2
```

```
3 int main(){
```

```
4  /* Escrivim un missatge per pantalla */
```

```
5  printf("Hello, World!\n");
```

```
6  return 0;
```

```
7 }
```

Aquesta línia no és codi, és un comentari. Tot el que estigui entre els símbols `/* */` no s'executarà

Estem cridant a una funció que es diu "printf" i que rep per paràmetre una cadena de text. Aquest procediment escriu per pantalla la cadena que nosaltres li passem per paràmetre.

El nostre primer programa

- Estructura d'aquest codi senzill

El programa es posa dins el procediment principal del programa (**main**). En un programa només hi ha un main. Les funcions que creem nosaltres podran tenir el nom que vulguem, però el main s'ha de dir main. El programa es comença a executar pel main.

Li diu al compilador que ens permeti fer servir les funcions incloses a la llibreria "**stdio.h**". Sense aquesta línia no es podria utilitzar el procediment **printf**, el compilador donaria un error.

```
1 #include <stdio.h>
```

```
2
```

```
3 int main(){
```

```
4  /* Escrivim un missatge per pantalla */
```

```
5  printf("Hello, World!\n");
```

```
6  return 0;
```

```
7 }
```

Aquesta línia no és codi, és un comentari. Tot el que estigui entre els símbols `/* */` no s'executarà

Estem cridant a una funció que es diu "printf" i que rep per paràmetre una cadena de text. Aquest procediment escriu per pantalla la cadena que nosaltres li passem per paràmetre.

Les sentències acaben amb ";"

El nostre primer programa

- Estructura d'aquest codi senzill

El programa es posa dins el procediment principal del programa (**main**). En un programa només hi ha un main. Les funcions que creem nosaltres podran tenir el nom que vulguem, però el main s'ha de dir main. El programa es comença a executar pel main.

Li diu al compilador que ens permeti fer servir les funcions incloses a la llibreria "**stdio.h**". Sense aquesta línia no es podria utilitzar el procediment **printf**, el compilador donaria un error.

```
1 #include <stdio.h>
```

```
2
```

```
3 int main(){
```

```
4  /* Escrivim un missatge per pantalla */
```

```
5  printf("Hello, World!\n");
```

```
6  return 0;
```

```
7 }
```

Aquesta línia no és codi, és un comentari. Tot el que estigui entre els símbols `/* */` no s'executarà

Estem cridant a una funció que es diu "printf" i que rep per paràmetre una cadena de text. Aquest procediment escriu per pantalla la cadena que nosaltres li passem per paràmetre.

Les funcions retornen algun valor. En aquest cas, hem definit que main retorni un enter (int) i per tant, estem obligats a retornar un enter (en aquest cas, 0).

Les sentències acaben amb ";"

El nostre primer programa

- Estructura d'aquest codi senzill

El programa es posa dins el procediment principal del programa (**main**). En un programa només hi ha un main. Les funcions que creem nosaltres podran tenir el nom que vulguem, però el main s'ha de dir main. El programa es comença a executar pel main.

Les claus {} es fan servir per contenir blocs de codi. Compte amb deixar-les o amb col·locarles malament!

Les funcions retornen algun valor. En aquest cas, hem definit que main retorni un enter (int) i per tant, estem obligats a retornar un enter (en aquest cas, 0).

Li diu al compilador que ens permeti fer servir les funcions incloses a la llibreria "**stdio.h**". Sense aquesta línia no es podria utilitzar el procediment **printf**, el compilador donaria un error.

Aquesta línia no és codi, és un comentari. Tot el que estigui entre els símbols /* */ no s'executarà

Estem cridant a una funció que es diu "printf" i que rep per paràmetre una cadena de text. Aquest procediment escriu per pantalla la cadena que nosaltres li passem per paràmetre.

Les sentències acaben amb ";"

```
1 #include <stdio.h>
2
3 int main(){
4     /* Escrivim un missatge per pantalla */
5     printf("Hello, World!\n");
6     return 0;
7 }
```

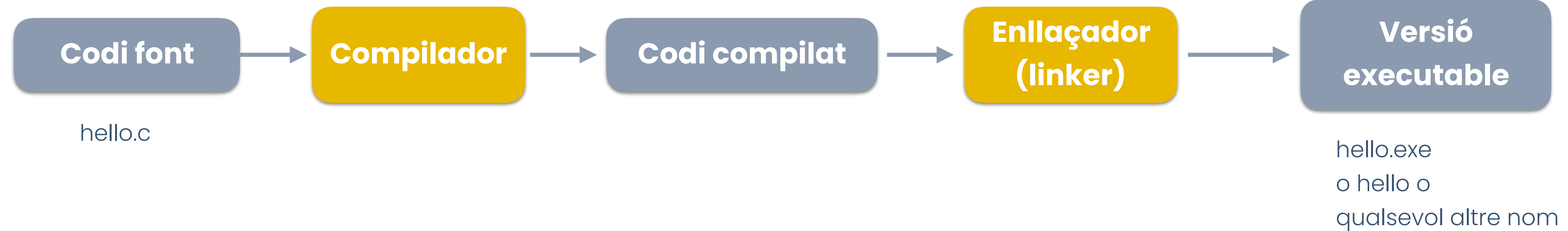

Compilació

- Molt bé, ja veig que per fer un programa he d'escriure un codi. Com el faig funcionar però???
- Necessitem compilar-lo. És a dir, traduir-lo de codi en llenguatge C a codi en llenguatge màquina.

Compilació

- Molt bé, ja veig que per fer un programa he d'escriure un codi. Com el faig funcionar però???
- Necessitem compilar-lo. És a dir, traduir-lo de codi en llenguatge C a codi en llenguatge màquina.

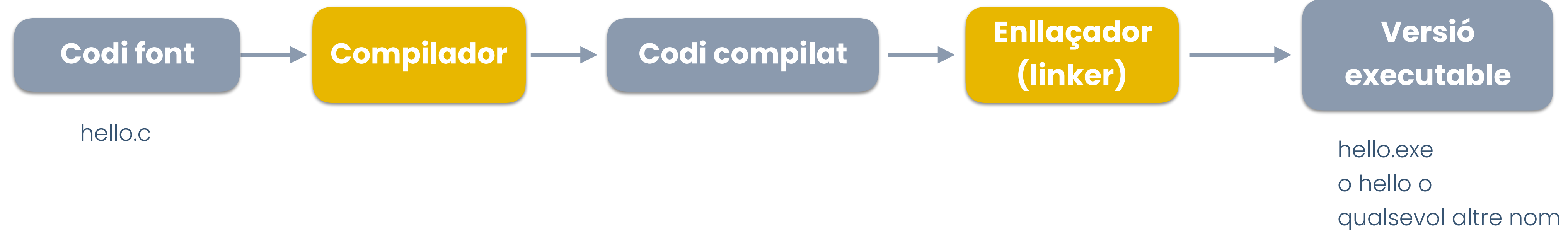
```
1 #include <stdio.h>
2
3 int main(){
4     printf("Hello, World!\n");
5     return 0;
6 }
```



Compilació

- Molt bé, ja veig que per fer un programa he d'escriure un codi. Com el faig funcionar però???
- Necessitem compilar-lo. És a dir, traduir-lo de codi en llenguatge C a codi en llenguatge màquina.

```
1 #include <stdio.h>
2
3 int main(){
4     printf("Hello, World!\n");
5     return 0;
6 }
```



```
○ ○ ○
> gcc hello.c -o hello
```

```
○ ○ ○
> gcc hello.c -o hello.exe
```


Com fer servir el terminal?

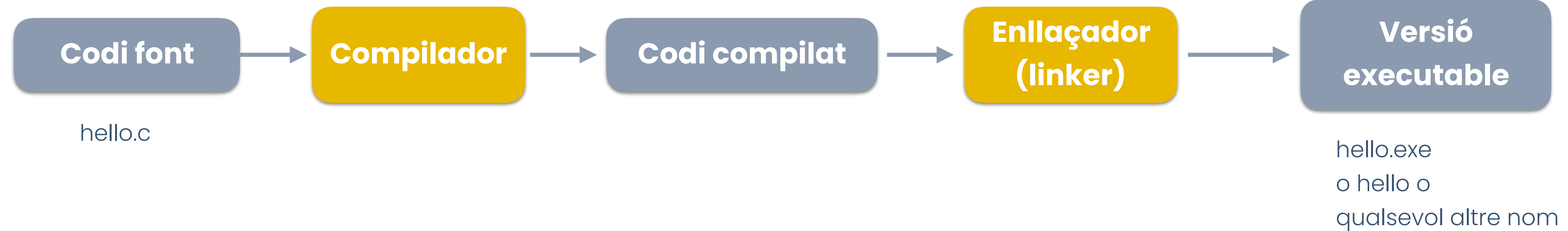
COMANDES FREQUENTS

UNIX/Mac OSX	Windows	
pwd	chdir	Saber on sóc ara mateix
ls	dir	Llistar arxius en la carpeta actual
mkdir "nom"	md "nom"	Crear directori nou anomenat "nom" a posició actual
cd "nom"	cd "nom"	Moure's a un directori que es diu "nom" i es troba on som
cd ..	cd ..	Tirar enrera en l'arbre de directoris
touch "nom"	No existeix	Crea un arxiu de text en blanc anomenat "nom"
clear	cls	Netejar pantalla
rm	del	Esborrar un fitxer (aneu amb compte!)

Compilació

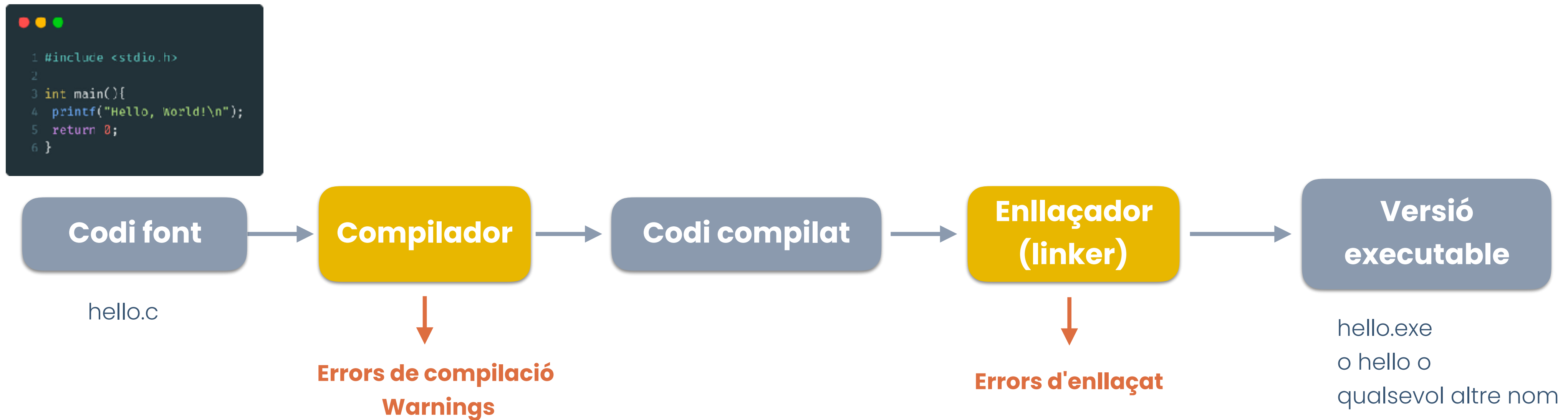
- Molt bé, ja veig que per fer un programa he d'escriure un codi. Com el faig funcionar però???
- Necessitem compilar-lo. És a dir, traduir-lo de codi en llenguatge C a codi en llenguatge màquina.

```
1 #include <stdio.h>
2
3 int main(){
4     printf("Hello, World!\n");
5     return 0;
6 }
```



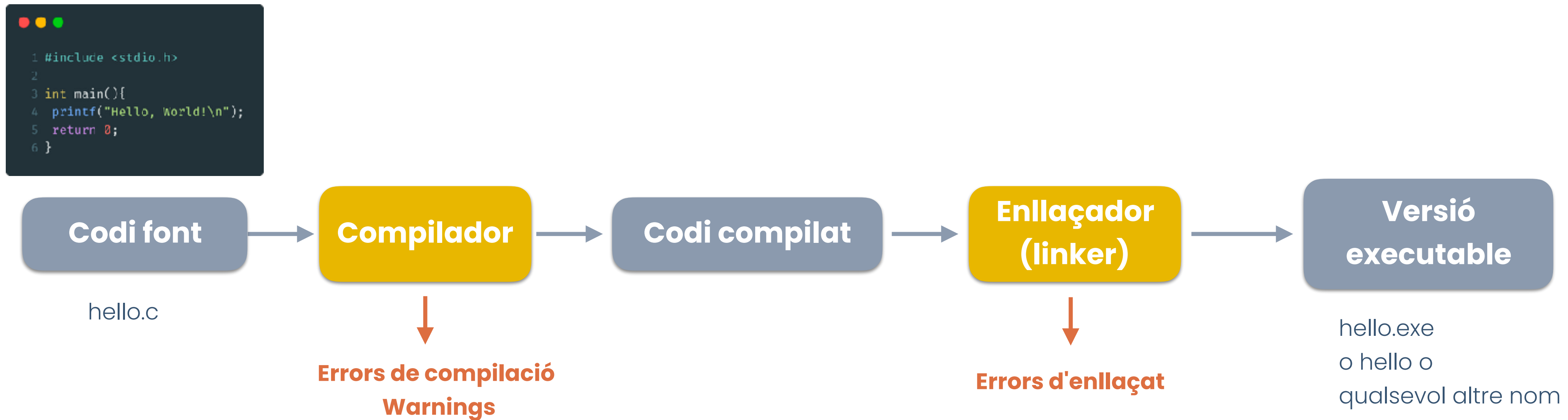
Compilació

- Molt bé, ja veig que per fer un programa he d'escriure un codi. Com el faig funcionar però???
- Necessitem compilar-lo. És a dir, traduir-lo de codi en llenguatge C a codi en llenguatge màquina.



Compilació

- Molt bé, ja veig que per fer un programa he d'escriure un codi. Com el faig funcionar però???
- Necessitem compilar-lo. És a dir, traduir-lo de codi en llenguatge C a codi en llenguatge màquina.



- En cas de trobar **errors** s'atura el procés i es mostra un missatge al programador. Per exemple, falta una clau, s'usa una variable que no s'ha definit...
- També poden aparèixer advertències (**warnings**), els quals no eviten obtenir l'executable, però avisen que hi ha alguna cosa que no està bé del tot i podria haver algun error mentre s'executa el programa.

Errors i warnings

- En el procés de compilació poden aparèixer errors i warnings.



```
1 #include <stdio.h>
2
3 int main()
4 {
5     /* Escrivim un missatge per pantalla */
6     printf("Hello, World!\n")
7     return 0;
8 }
```



```
1 #include <stdio.h>
2
3 main()
4 {
5     /* Escrivim un missatge per pantalla */
6     printf("Hello, World!\n");
7     return 0;
8 }
```


Errors i warnings

- En el procés de compilació poden aparèixer errors i warnings.

Errors de compilació

- No permeten que el programa es compili, per tant no tenim res a executar
- Són una gran ajuda: millor un error ara que un malfuncionament després.



```
1 #include <stdio.h>
2
3 int main()
4 {
5     /* Escrivim un missatge per pantalla */
6     printf("Hello, World!\n")
7     return 0;
8 }
```



```
1 #include <stdio.h>
2
3 main()
4 {
5     /* Escrivim un missatge per pantalla */
6     printf("Hello, World!\n");
7     return 0;
8 }
```

Errors i warnings

- En el procés de compilació poden aparèixer errors i warnings.

Errors de compilació

- No permeten que el programa es compili, per tant no tenim res a executar
- Són una gran ajuda: millor un error ara que un malfuncionament després.



```
1 #include <stdio.h>
2
3 int main()
4 {
5     /* Escrivim un missatge per pantalla */
6     printf("Hello, World!\n")
7     return 0;
8 }
```

Error creat a propòsit: falta el ";"



```
1 #include <stdio.h>
2
3 main()
4 {
5     /* Escrivim un missatge per pantalla */
6     printf("Hello, World!\n");
7     return 0;
8 }
```


Errors i warnings

- En el procés de compilació poden aparèixer errors i warnings.

Errors de compilació

- No permeten que el programa es compili, per tant no tenim res a executar
- Són una gran ajuda: millor un error ara que un malfuncionament després.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     /* Escrivim un missatge per pantalla */
6     printf("Hello, World!\n")
7     return 0;
8 }
```

Error creat a propòsit: falta el ";"

```
1 #include <stdio.h>
2
3 main()
4 {
5     /* Escrivim un missatge per pantalla */
6     printf("Hello, World!\n");
7     return 0;
8 }
```

hello.c:6:27: **error:** expected ';' after expression

Errors i warnings

- En el procés de compilació poden aparèixer errors i warnings.

Errors de compilació

- No permeten que el programa es compili, per tant no tenim res a executar
- Són una gran ajuda: millor un error ara que un malfuncionament després.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     /* Escrivim un missatge per pantalla */
6     printf("Hello, World!\n")
7     return 0;
8 }
```

Error creat a propòsit: falta el ";"

```
hello.c:6:27: error: expected ';' after expression
```

Warnings

- Són avisos de possibles malfuncionaments. Permet compilar el programa i executar-lo.
- No s'han de deixar passar, nosaltres els tractarem com a errors. Si ens avisa d'un possible problema, l'hem de resoldre.

```
1 #include <stdio.h>
2
3 main()
4 {
5     /* Escrivim un missatge per pantalla */
6     printf("Hello, World!\n");
7     return 0;
8 }
```

Errors i warnings

- En el procés de compilació poden aparèixer errors i warnings.

Errors de compilació

- No permeten que el programa es compili, per tant no tenim res a executar
- Són una gran ajuda: millor un error ara que un malfuncionament després.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     /* Escrivim un missatge per pantalla */
6     printf("Hello, World!\n")
7     return 0;
8 }
```

Error creat a propòsit: falta el ";"

```
hello.c:6:27: error: expected ';' after expression
```

Warnings

- Són avisos de possibles malfuncionaments. Permet compilar el programa i executar-lo.
- No s'han de deixar passar, nosaltres els tractarem com a errors. Si ens avisa d'un possible problema, l'hem de resoldre.

```
1 #include <stdio.h>
2
3 main()
4 {
5     /* Escrivim un missatge per pantalla */
6     printf("Hello, World!\n");
7     return 0;
8 }
```

Warning creat a propòsit: ens hem oblidat d'especificar el valor de retorn de la funció main.

Errors i warnings

- En el procés de compilació poden aparèixer errors i warnings.

Errors de compilació

- No permeten que el programa es compili, per tant no tenim res a executar
- Són una gran ajuda: millor un error ara que un malfuncionament després.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     /* Escrivim un missatge per pantalla */
6     printf("Hello, World!\n")
7     return 0;
8 }
```

Error creat a propòsit: falta el ";"

```
hello.c:6:27: error: expected ';' after expression
```

Warnings

- Són avisos de possibles malfuncionaments. Permet compilar el programa i executar-lo.
- No s'han de deixar passar, nosaltres els tractarem com a errors. Si ens avisa d'un possible problema, l'hem de resoldre.

```
1 #include <stdio.h>
2
3 main()
4 {
5     /* Escrivim un missatge per pantalla */
6     printf("Hello, World!\n");
7     return 0;
8 }
```

Warning creat a propòsit: ens hem oblidat d'especificar el valor de retorn de la funció main.

```
hello.c:3:1: warning: type specifier missing, defaults to 'int' [-Wimplicit-int]
```

Errors i warnings

- En el procés de compilació poden aparèixer errors i warnings.

Errors de compilació

- No permeten que el programa es compili, per tant no tenim res a executar
- Són una gran ajuda: millor un error ara que un malfuncionament després.

Warnings

- Són avisos de possibles malfuncionaments. Permet compilar el programa i executar-lo.
- No s'han de deixar passar, nosaltres els tractarem com a errors. Si ens avisa d'un possible problema, l'hem de resoldre.

Errors i warnings

- En el procés de compilació poden aparèixer errors i warnings.

Errors de compilació

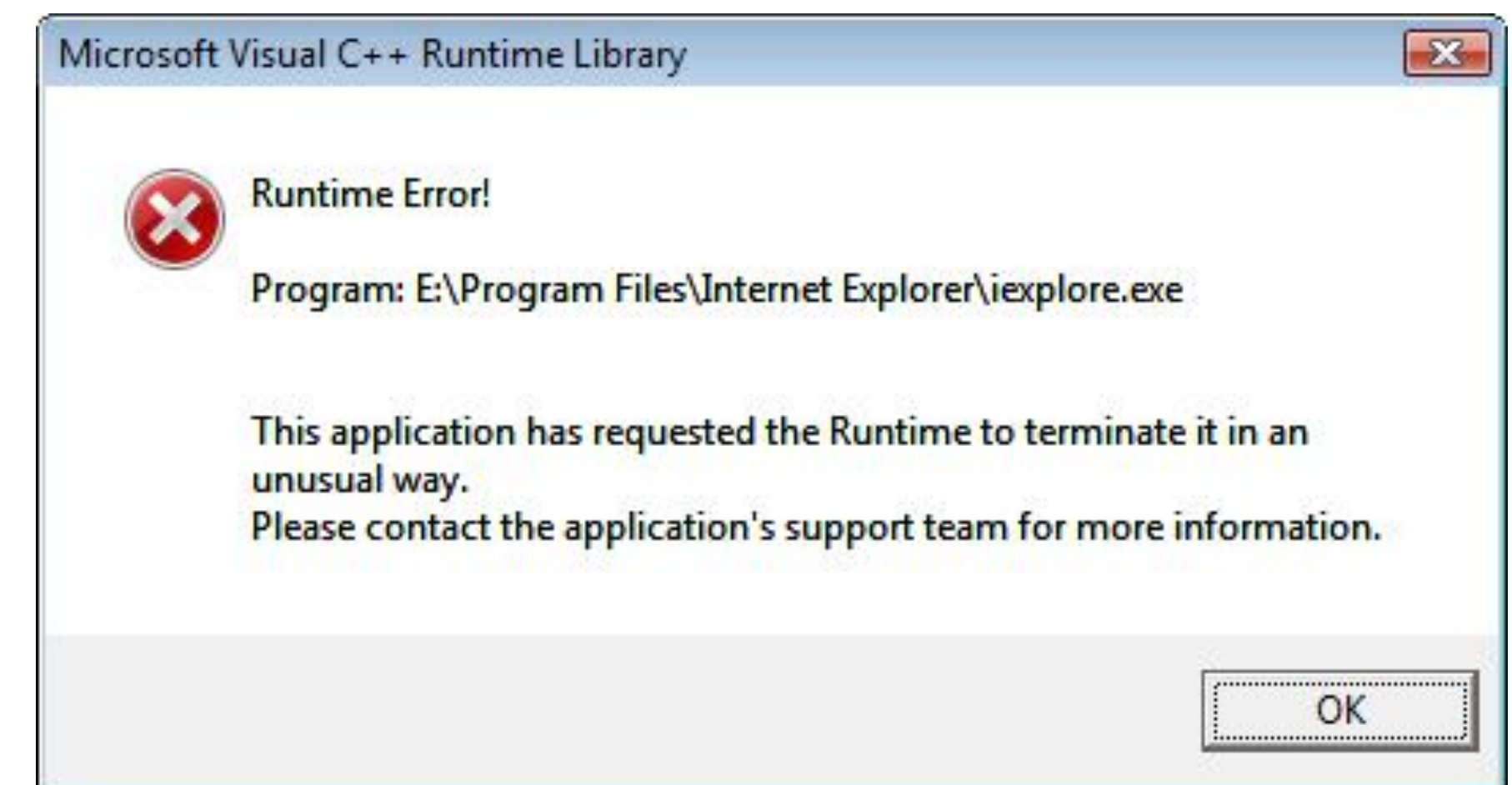
- No permeten que el programa es compili, per tant no tenim res a executar
- Són una gran ajuda: millor un error ara que un malfuncionament després.

Errors en temps d'execució

- Un tipus d'errors pitjors que els de passen durant el procés de compilació perquè no ens donen temps a reaccionar.
- En aquest cas el programa compila perfectament però es produeix algun error mentre s'executa el programa i el deixen sense funcionament. A vegades és un "warning" al qual no hem fet cas...
- Per exemple: si intento accedir a una posició de memòria que no toca, etc...

Warnings

- Són avisos de possibles malfuncionaments. Permet compilar el programa i executar-lo.
- No s'han de deixar passar, nosaltres els tractarem com a errors. Si ens avisa d'un possible problema, l'hem de resoldre.



SINTAXI BÀSICA

Sintaxi bàsica

```
#include <stdio.h>
#include <stdlib.h>

// Això és un comentari d'una línia

/* Això és un bloc
de comentari multilínia*/

int main() {

    int a, b; // Declaro a i b

    a = 0;
    b=1;
    a = a + b;

    if (a < b){
        printf("A menor que B");
    }
    else {
        printf("Va a ser que no");
    }
    return 0;
}
```

Sintaxi bàsica

Semicolons

- En C, qualsevol instrucció ha d'acabar amb un punt i coma ";". Si no és així, hi haurà un error de compilació.

```
#include <stdio.h>
#include <stdlib.h>

// Això és un comentari d'una línia

/* Això és un bloc
de comentari multilínia*/

int main() {

    int a, b; // Declaro a i b

    a = 0;
    b=1;
    a = a + b;

    if (a < b){
        printf("A menor que B");
    }
    else {
        printf("Va a ser que no");
    }
    return 0;
}
```


Sintaxi bàsica

Semicolons

- En C, qualsevol instrucció ha d'acabar amb un punt i coma ";". Si no és així, hi haurà un error de compilació.

Comentaris

- Hi ha dos tipus de comentaris: els d'una línia, precedits per `//`, i el bloc de comentaris, que va entre `/*` i `*/`
- Nota: els comentaris `//` no formen part de l'estàndard ANSI C, sinó de C99

```
#include <stdio.h>
#include <stdlib.h>

// Això és un comentari d'una línia

/* Això és un bloc
de comentari multilínia*/

int main() {

    int a, b; // Declaro a i b

    a = 0;
    b=1;
    a = a + b;

    if (a < b){
        printf("A menor que B");
    }
    else {
        printf("Va a ser que no");
    }
    return 0;
}
```

Sintaxi bàsica

Semicolons

- En C, qualsevol instrucció ha d'acabar amb un punt i coma ";". Si no és així, hi haurà un error de compilació.

Comentaris

- Hi ha dos tipus de comentaris: els d'una línia, precedits per `//`, i el bloc de comentaris, que va entre `/*` i `*/`
- Nota: els comentaris `//` no formen part de l'estàndard ANSI C, sinó de C99

Assignacions

- A teoria, en pseudocodi, usem `:=` per assignar. En C, es fa servir simplement `=`.

```
#include <stdio.h>
#include <stdlib.h>

// Això és un comentari d'una línia

/* Això és un bloc
de comentari multilínia*/

int main() {

    int a, b; // Declaro a i b

    a = 0;
    b=1;
    a = a + b;

    if (a < b){
        printf("A menor que B");
    }
    else {
        printf("Va a ser que no");
    }
    return 0;
}
```

Sintaxi bàsica

Semicolons

- En C, qualsevol instrucció ha d'acabar amb un punt i coma ";". Si no és així, hi haurà un error de compilació.

Comentaris

- Hi ha dos tipus de comentaris: els d'una línia, precedits per `//`, i el bloc de comentaris, que va entre `/*` i `*/`
- Nota: els comentaris `//` no formen part de l'estàndard ANSI C, sinó de C99

Assignacions

- A teoria, en pseudocodi, usem `:=` per assignar. En C, es fa servir simplement `=`.

Espais en blanc

- Els espais en blanc no importen (a no ser que divideixin una paraula!). Es poden afegir per motius de llegibilitat.

```
#include <stdio.h>
#include <stdlib.h>

// Això és un comentari d'una línia

/* Això és un bloc
de comentari multilínia*/

int main() {

    int a, b; // Declaro a i b

    a = 0;
    b=1;
    a = a + b;

    if (a < b){
        printf("A menor que B");
    }
    else {
        printf("Va a ser que no");
    }
    return 0;
}
```


Sintaxi bàsica

```
#include <stdio.h>
#include <stdlib.h>

// Això és un comentari d'una línia

/* Això és un bloc
de comentari multilínia*/

int main() {

    int a, b; // Declaro a i b

    a = 0;
    b=1;
    a = a + b;

    if (a < b){
        printf("A menor que B");
    }
    else {
        printf("Va a ser que no");
    }
    return 0;
}
```

Sintaxi bàsica

Identificadors

- Un identificador és el nom que usem per identificar una funció, variable o qualsevol altre ítem que defineix l'usuari. Poden contenir lletres, números, underscore.
- No es permeten caràcters @, \$ i % en els identificadors
- C és **case sensitive**: num i Num són diferents.
- Res de caràcters amb accents o ç o ñ. (Només ASCII estàndard)

```
#include <stdio.h>
#include <stdlib.h>

// Això és un comentari d'una línia

/* Això és un bloc
de comentari multilínia*/

int main() {

    int a, b; // Declaro a i b

    a = 0;
    b=1;
    a = a + b;

    if (a < b){
        printf("A menor que B");
    }
    else {
        printf("Va a ser que no");
    }
    return 0;
}
```

Sintaxi bàsica

Identificadors

- Un identificador és el nom que usem per identificar una funció, variable o qualsevol altre ítem que defineix l'usuari. Poden contenir lletres, números, underscore.
- No es permeten caràcters @, \$ i % en els identificadors
- C és **case sensitive**: num i Num són diferents.
- Res de caràcters amb accents o ç o ñ. (Només ASCII estàndard)

Paraules reservades (keywords)

- Són paraules que tenen un significat concret en C i no podem fer-les servir com a identificadors.

auto	case	const	default
else	extern	for	if
long	return	signed	static
switch	union	void	while
break	char	continue	do
enum	float	goto	int
register	short	sizeof	struct
typedef	unsigned	volatile	_Packed
			double

```
#include <stdio.h>
#include <stdlib.h>

// Això és un comentari d'una línia

/* Això és un bloc
de comentari multilínia*/

int main() {

    int a, b; // Declaro a i b

    a = 0;
    b=1;
    a = a + b;

    if (a < b){
        printf("A menor que B");
    }
    else {
        printf("Va a ser que no");
    }
    return 0;
}
```


Sintaxi bàsica

La importància de la indentació

- La indentació significa marcar l'estructura del codi fent servir espais o tabuladors.
- En alguns llenguatges de programació, la indentació és obligatòria: la manera de separar blocs es fa amb indentació
- En C no és obligatori perquè funcioni però sí que ho és perquè s'entengui.

Sintaxi bàsica

La importància de la indentació

```
int main() {  
  
    int * taula;  
    int n = 10;  
    FILE *f_out;  
  
    taula = (int *) malloc(n * sizeof(int));  
  
    if (usuari == 1) {  
        printf("Introdueix %d enters.\n",n);  
        for(int i=0; i<n; i++){  
            scanf("%d", &taula[i]);  
        }  
    }  
    return 0;  
}
```

Codi ben indentat: a simple vista podem veure
l'estructura de blocs

```
int main() {  
  
int * taula;  
int n = 10;  
FILE *f_out;  
  
    taula = (int *) malloc(n * sizeof(int));  
  
    if (usuari == 1) {  
        printf("Introdueix %d enters.\n",n);  
        for(int i=0; i<n; i++){  
            scanf("%d", &taula[i]);  
        }  
    }  
    return 0;  
}
```

Codi mal indentat: no podem veure clarament
l'estructura de blocs

VARIABLES | CONSTANTS

Variables en C

A teoria:

```
var
    pes: enter;
    index_MC: real;
    c1, c2: caràcter;
fvar
```

Variables en C

A teoria:

```
var
    pes: enter;
    index_MC: real;
    c1, c2: caràcter;
fvar
```

En C:

- No hi ha bloc **var** ... **fvar** però seguirem declarant les variables sempre al principi.
- Les variables s'han de declarar abans d'usar-les.
- Primer s'especifica el tipus i després el nom de la variable.

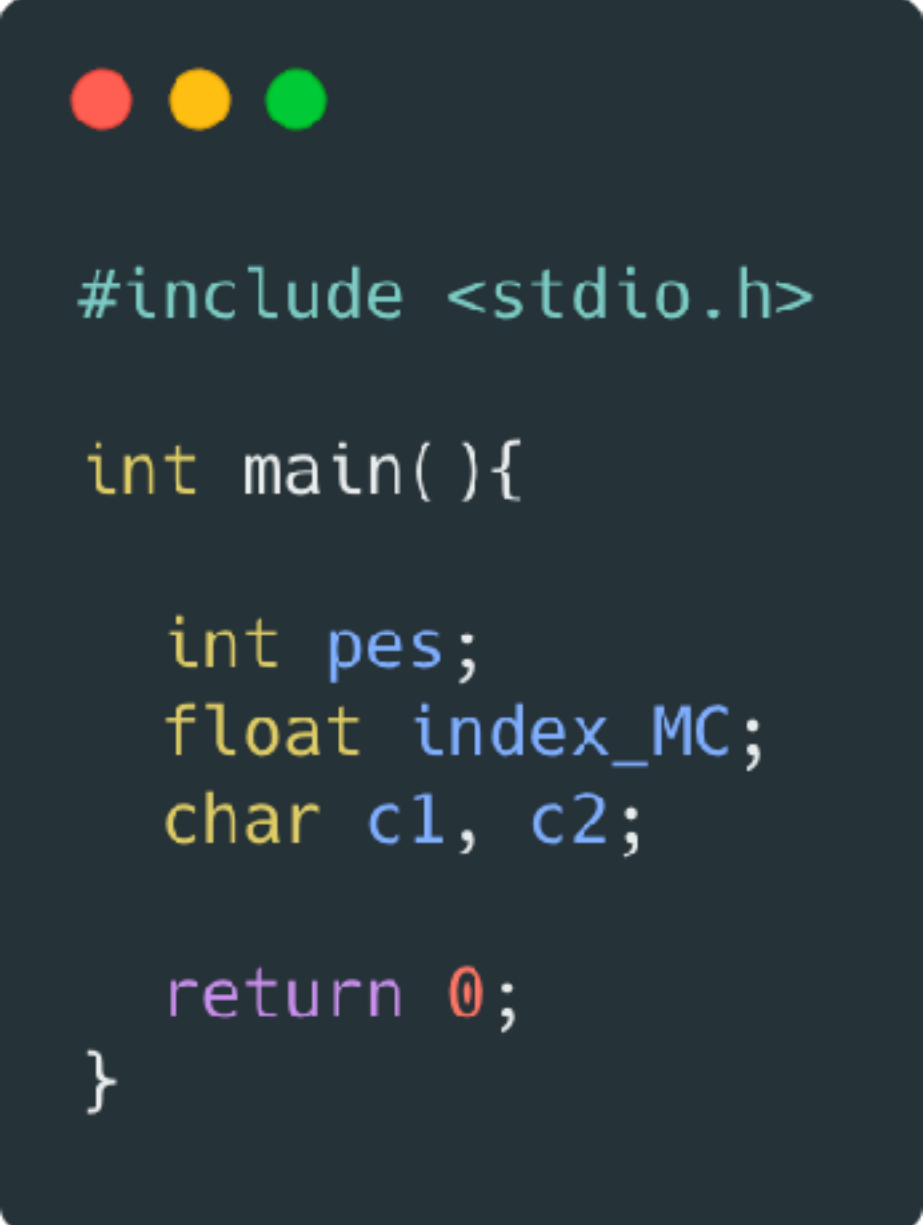
Variables en C

A teoria:

```
var
    pes: enter;
    index_MC: real;
    c1, c2: caràcter;
fvar
```

En C:

- No hi ha bloc **var** ... **fvar** però seguirem declarant les variables sempre al principi.
- Les variables s'han de declarar abans d'usar-les.
- Primer s'especifica el tipus i després el nom de la variable.



```
#include <stdio.h>

int main(){

    int pes;
    float index_MC;
    char c1, c2;

    return 0;
}
```


Tipus de variables

Numèriques (enteres)

Type	Storage size	Value range
char	1 byte	-128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
signed char	1 byte	-128 to 127
int	2 or 4 bytes	-32,768 to 32,767 or -2,147,483,648 to 2,147,483,647
unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295
short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
long	8 bytes or (4bytes for 32 bit OS)	-9223372036854775808 to 9223372036854775807
unsigned long	8 bytes	0 to 18446744073709551615

Tipus de variables

Numèriques (enteres)

Type	Storage size	Value range
char	1 byte	-128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
signed char	1 byte	-128 to 127
int	2 or 4 bytes	-32,768 to 32,767 or -2,147,483,648 to 2,147,483,647
unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295
short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
long	8 bytes or (4bytes for 32 bit OS)	-9223372036854775808 to 9223372036854775807
unsigned long	8 bytes	0 to 18446744073709551615

Numèriques (decimals)

Type	Storage size	Value range	Precision
float	4 byte	1.2E-38 to 3.4E+38	6 decimal places
double	8 byte	2.3E-308 to 1.7E+308	15 decimal places
long double	10 byte	3.4E-4932 to 1.1E+4932	19 decimal places

Tipus de variables

Numèriques (enteres)

Type	Storage size	Value range
char	1 byte	-128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
signed char	1 byte	-128 to 127
int	2 or 4 bytes	-32,768 to 32,767 or -2,147,483,648 to 2,147,483,647
unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295
short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
long	8 bytes or (4bytes for 32 bit OS)	-9223372036854775808 to 9223372036854775807
unsigned long	8 bytes	0 to 18446744073709551615

Numèriques (decimals)

Type	Storage size	Value range	Precision
float	4 byte	1.2E-38 to 3.4E+38	6 decimal places
double	8 byte	2.3E-308 to 1.7E+308	15 decimal places
long double	10 byte	3.4E-4932 to 1.1E+4932	19 decimal places

Variables booleanes

- En el primer estàndard de C (ANSI C) el tipus booleà no existeix. S’havien d’implementar per mitjà de nombres enters, on 0 és fals i diferent de 0 és cert.

```
#include <stdio.h>

int main(){

    int trobat = 0;

    return 0;
}
```


Tipus de variables

Numèriques (enteres)

Type	Storage size	Value range
char	1 byte	-128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
signed char	1 byte	-128 to 127
int	2 or 4 bytes	-32,768 to 32,767 or -2,147,483,648 to 2,147,483,647
unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295
short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
long	8 bytes or (4bytes for 32 bit OS)	-9223372036854775808 to 9223372036854775807
unsigned long	8 bytes	0 to 18446744073709551615

Numèriques (decimals)

Type	Storage size	Value range	Precision
float	4 byte	1.2E-38 to 3.4E+38	6 decimal places
double	8 byte	2.3E-308 to 1.7E+308	15 decimal places
long double	10 byte	3.4E-4932 to 1.1E+4932	19 decimal places

Variables booleanes

- En el primer estàndard de C (ANSI C) el tipus booleà no existeix. S’havien d’implementar per mitjà de nombres enters, on 0 és fals i diferent de 0 és cert.

```
#include <stdio.h>

int main(){
    int trobat = 0;

    return 0;
}
```

- En llenguatge C actual podem definir el tipus booleà i usar els valors “true” i “false”, sempre i quan incloguem la llibreria **<stdbool.h>**

```
#include <stdio.h>
#include <stdbool.h>

int main(){
    bool trobat;
    bool final;
    trobat = true;
    final = false;
    return 0;
}
```

Resum de tipus de variables

En pseudocodi	En C		
	Tipus	Mida	Domini o Rang
enter	short ¹	(2 bytes) 16 bits	-32768 ... 32767
	int ¹	(4 bytes) 32 bits	-2147483648...2147483647
	long ¹	(8 bytes) 64 bits	-9223372036854775808... 9223372036854775807
real	float	(4 bytes) 32 bits	-3.4E+38 ... 3.4E+38
	double	(8 bytes) 64 bits	-1.7E-308 ... 1.7E+308
	long double	(16 bytes) 128 bits	-3.4E-4092 ... 3.4E+4092
booleà	bool	(1 byte) 8 bits	true (≠0), false (0)
caràcter	char ¹	(1 byte) 8 bits	-127 ... 128

¹La paraula **unsigned** es pot afegir al tipus.

Constants en C

A teoria:

```
const
    MAX_ELEMS:= 100;
    N_FILES:= 1000;
    PI:= 3.1415;
fconst
```


Constants en C

A teoria:

```
const
    MAX_ELEMS := 100;
    N_FILES := 1000;
    PI := 3.1415;
fconst
```

En C:

- Valors literals: són valors immutables: “hola”, ‘A’, 3.14, 1000... etc
- Hi ha dues maneres de definir constants en C:
 - “Convertir” una variable en una constant:
 - Definint una macro de preprocessador

Constants en C

A teoria:

```
const
    MAX_ELEMS := 100;
    N_FILES := 1000;
    PI := 3.1415;
fconst
```

En C:

- Valors literals: són valors immutables: "hola", 'A', 3.14, 1000... etc
- Hi ha dues maneres de definir constants en C:
 - "Convertir" una variable en una constant:
 - Definint una macro de preprocessador

Opció 1: "const"

- Només cal afegir la paraula reservada **const** davant la declaració de la variable.
- A tots els efectes, és una variable, però de la qual no es pot canviar el valor un cop assignada.

Constants en C

A teoria:

const

```
MAX_ELEMS:= 100;  
N_FILES:= 1000;  
PI:= 3.1415;
```

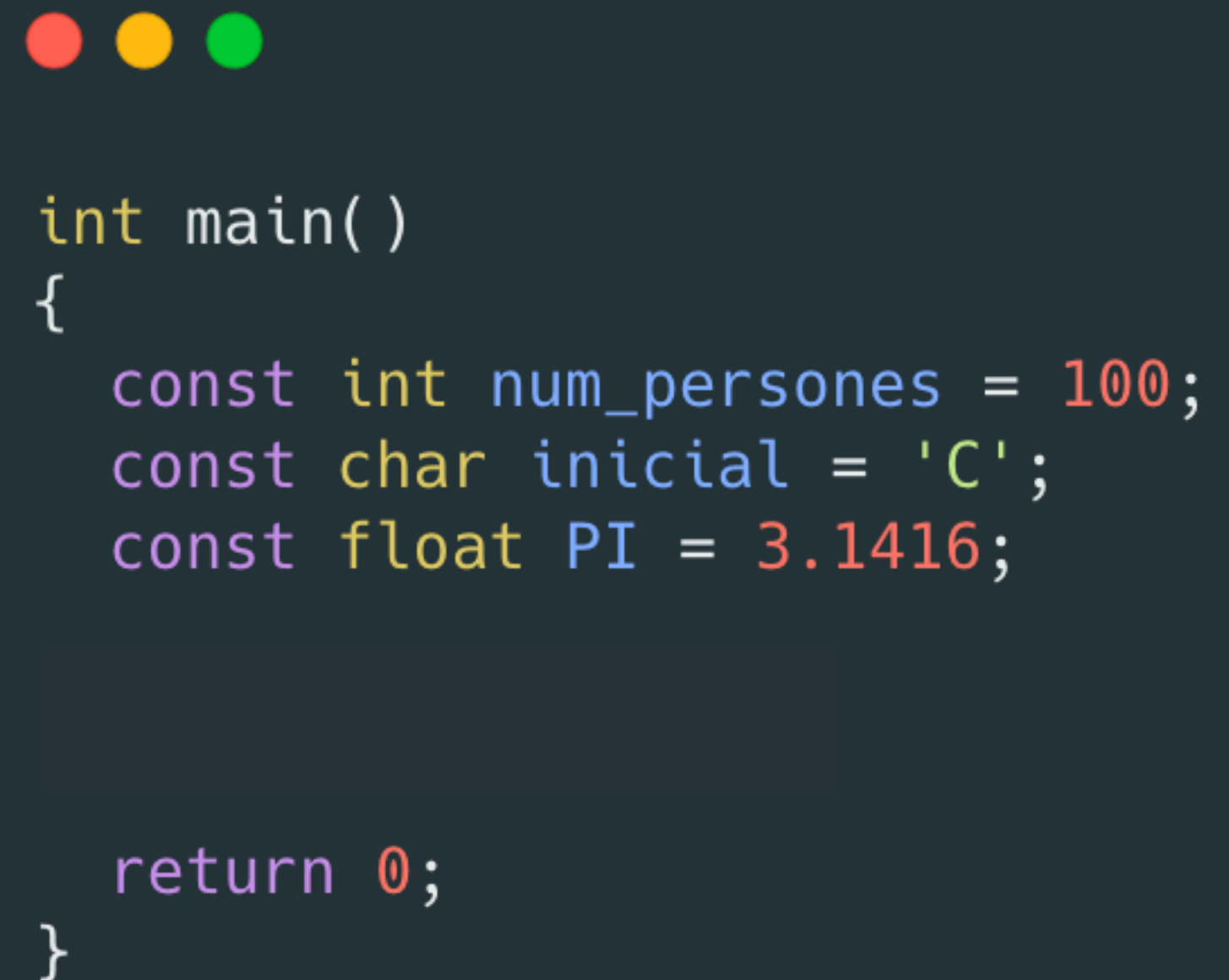
fconst

En C:

- Valors literals: són valors immutables: "hola", 'A', 3.14, 1000... etc
- Hi ha dues maneres de definir constants en C:
 - "Convertir" una variable en una constant:
 - Definint una macro de preprocessador

Opció 1: "const"

- Només cal afegir la paraula reservada **const** davant la declaració de la variable.
- A tots els efectes, és una variable, però de la qual no es pot canviar el valor un cop assignada.



```
int main()  
{  
    const int num_persones = 100;  
    const char inicial = 'C';  
    const float PI = 3.1416;  
  
    return 0;  
}
```


Constants en C

A teoria:

const

```
MAX_ELEMS:= 100;  
N_FILES:= 1000;  
PI:= 3.1415;
```

fconst

En C:

- Valors literals: són valors immutables: "hola", 'A', 3.14, 1000... etc
- Hi ha dues maneres de definir constants en C:
 - "Convertir" una variable en una constant:
 - Definint una macro de preprocessador

Opció 1: "const"

- Només cal afegir la paraula reservada **const** davant la declaració de la variable.
- A tots els efectes, és una variable, però de la qual no es pot canviar el valor un cop assignada.

```
int main()  
{  
    const int num_persones = 100;  
    const char inicial = 'C';  
    const float PI = 3.1416;  
  
    PI = 23; // Error  
  
    return 0;  
}
```

constants.c:9:6: error: assignment of read-only variable 'PI'

Constants en C

A teoria:

```
const
    MAX_ELEMS := 100;
    N_FILES := 1000;
    PI := 3.1415;
fconst
```

En C:

- Valors literals: són valors immutables: “hola”, ‘A’, 3.14, 1000... etc
- Hi ha dues maneres de definir constants en C:
 - “Convertir” una variable en una constant:
 - Definint una macro de preprocessador

Constants en C

A teoria:

```
const
    MAX_ELEMS := 100;
    N_FILES := 1000;
    PI := 3.1415;
fconst
```

En C:

- Valors literals: són valors immutables: "hola", 'A', 3.14, 1000... etc
- Hi ha dues maneres de definir constants en C:
 - "Convertir" una variable en una constant:
 - Definint una macro de preprocessador

Opció 2: "#define"

- La paraula reservada #define associa a una paraula clau un valor concret.
- En temps de compilació, el compilador substitueix totes les ocurrències d'aquesta paraula dins del codi. Per tant, no és una variable en memòria.

Constants en C

A teoria:

const

```
MAX_ELEMS:= 100;  
N_FILES:= 1000;  
PI:= 3.1415;
```

fconst

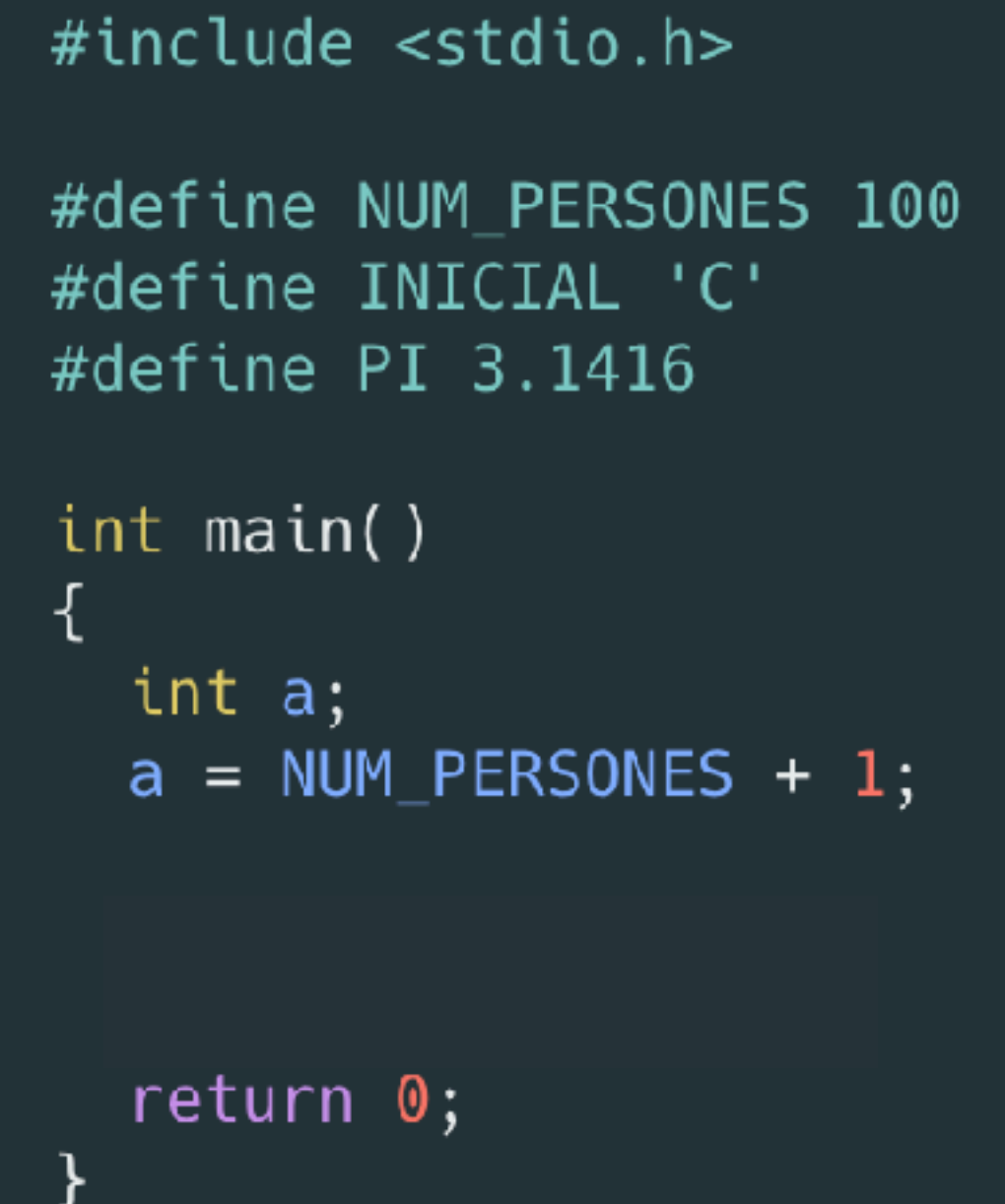
En C:

- Valors literals: són valors immutables: "hola", 'A', 3.14, 1000... etc
- Hi ha dues maneres de definir constants en C:
 - "Convertir" una variable en una constant:
 - Definint una macro de preprocessador

Opció 2: "#define"

- La paraula reservada #define associa a una paraula clau un valor concret.
- En temps de compilació, el compilador substitueix totes les ocurrències d'aquesta paraula dins del codi. Per tant, no és una variable en memòria.

Els "define" es declaren a la capçalera, no tenen operador d'assignació "=" i no acaben amb ";"



```
#include <stdio.h>  
  
#define NUM_PERSONES 100  
#define INICIAL 'C'  
#define PI 3.1416  
  
int main()  
{  
    int a;  
    a = NUM_PERSONES + 1;  
  
    return 0;  
}
```

Constants en C

A teoria:

const

```
MAX_ELEMS := 100;  
N_FILES := 1000;  
PI := 3.1415;
```

fconst

En C:

- Valors literals: són valors immutables: "hola", 'A', 3.14, 1000... etc
- Hi ha dues maneres de definir constants en C:
 - "Convertir" una variable en una constant:
 - Definint una macro de preprocessador

Opció 2: "#define"

- La paraula reservada #define associa a una paraula clau un valor concret.
- En temps de compilació, el compilador substitueix totes les ocurrències d'aquesta paraula dins del codi. Per tant, no és una variable en memòria.

Els "define" es declaren a la capçalera, no tenen operador d'assignació "=" i no acaben amb ";"

```
#include <stdio.h>  
  
#define NUM_PERSONES 100  
#define INICIAL 'C'  
#define PI 3.1416  
  
int main()  
{  
    int a;  
    a = NUM_PERSONES + 1;  
  
    PI = 400; // Error  
  
    return 0;  
}
```

constants.c:12:16: error: lvalue required as left operand of assignment

LLEGIR I ESCRIURE

PER L'ENTRADA/SORTIDA ESTÀNDARD

Acte de fe



Qui no entengui aquesta referència està suspès

Escriure per pantalla

A teoria:

```
...  
escriure("Hola!");  
escriure("El valor de N és", N);  
...
```

Escriure per pantalla

A teoria:

```
...  
escriure("Hola!");  
escriure("El valor de N és", N);  
...
```

En C:

- Farem servir la funció **printf()**, disponible a la llibreria **<stdio.h>** (llibreria d'entrada i sortida I/O)
<https://en.cppreference.com/w/c/io>

```
int printf(const char *format, ...)
```

- Envia una sortida formatejada a la sortida estàndard (stdout, o sigui, la pantalla)
- No inclou un salt de línia per defecte

Escriure per pantalla

A teoria:

```
...
escriure("Hola!");
escriure("El valor de N és", N);
...
```

En C:

- Farem servir la funció `printf()`, disponible a la llibreria `<stdio.h>` (llibreria d'entrada i sortida I/O)
<https://en.cppreference.com/w/c/io>

```
int printf(const char *format, ...)
```

- Envia una sortida formatejada a la sortida estàndard (stdout, o sigui, la pantalla)
- No inclou un salt de línia per defecte

Caràcters de format més freqüents:

<code>\a</code>	audible alert (bell)	<code>\</code>	escape the next character
<code>\b</code>	backspace	<code>\\</code>	print a backslash
<code>\f</code>	form feed	<code>"</code>	start or end of string
<code>\n</code>	newline (linefeed)	<code>\"</code>	print a double quote
<code>\r</code>	carriage return	<code>'</code>	start or end a character constant
<code>\t</code>	tab	<code>\'</code>	print a single quote
<code>\v</code>	vertical tab	<code>%</code>	start a format specification
		<code>\%</code>	print a percent sign

Caràcters especials

Escriure per pantalla

A teoria:

```
...
escriure("Hola!");
escriure("El valor de N és", N);
...
```

En C:

- Farem servir la funció `printf()`, disponible a la llibreria `<stdio.h>` (llibreria d'entrada i sortida I/O)
<https://en.cppreference.com/w/c/io>

```
int printf(const char *format, ...)
```

- Envia una sortida formatejada a la sortida estàndard (stdout, o sigui, la pantalla)
- No inclou un salt de línia per defecte

Caràcters de format més freqüents:

\a	audible alert (bell)	\	escape the next character
\b	backspace	\\	print a backslash
\f	form feed	"	start or end of string
\n	newline (linefeed)	\"	print a double quote
\r	carriage return	'	start or end a character constant
\t	tab	\'	print a single quote
\v	vertical tab	%	start a format specification
		\%	print a percent sign

Caràcters especials

%c	print a single character
%d	print a decimal (base 10) number
%e	print an exponential floating-point number
%f	print a floating-point number
%g	print a general-format floating-point number
%i	print an integer in base 10
%o	print a number in octal (base 8)
%s	print a string of characters
%u	print an unsigned decimal (base 10) number
%x	print a number in hexadecimal (base 16)
%%	print a percent sign (\% also works)

Caràcters de format.

Cada especificació de format comença amb el símbol % i acaba en una lletra

Escriure per pantalla

A teoria:

```
...
escriure("Hola!");
escriure("El valor de N és", N);
...
```

En C:

- Farem servir la funció `printf()`, disponible a la llibreria `<stdio.h>` (llibreria d'entrada i sortida I/O) <https://en.cppreference.com/w/c/io>

```
int printf(const char *format, ...)
```

- Envia una sortida formatejada a la sortida estàndard (stdout, o sigui, la pantalla)
- No inclou un salt de línia per defecte

Caràcters de format més freqüents:

<code>\a</code>	audible alert (bell)	<code>\</code>	escape the next character
<code>\b</code>	backspace	<code>\\</code>	print a backslash
<code>\f</code>	form feed	<code>"</code>	start or end of string
<code>\n</code>	newline (linefeed)	<code>\"</code>	print a double quote
<code>\r</code>	carriage return	<code>'</code>	start or end a character constant
<code>\t</code>	tab	<code>\'</code>	print a single quote
<code>\v</code>	vertical tab	<code>%</code>	start a format specification
		<code>\%</code>	print a percent sign

Caràcters especials

<code>%c</code>	print a single character
<code>%d</code>	print a decimal (base 10) number
<code>%e</code>	print an exponential floating-point number
<code>%f</code>	print a floating-point number
<code>%g</code>	print a general-format floating-point number
<code>%i</code>	print an integer in base 10
<code>%o</code>	print a number in octal (base 8)
<code>%s</code>	print a string of characters
<code>%u</code>	print an unsigned decimal (base 10) number
<code>%x</code>	print a number in hexadecimal (base 16)
<code>%%</code>	print a percent sign (<code>\%</code> also works)

Caràcters de format.

Cada especificació de format comença amb el símbol `%` i acaba en una lletra

<code>("%.1f",e)</code>	2.7
<code>("%.2f",e)</code>	2.72
<code>("%.6f",e)</code>	2.718282

Posicions decimals

Per practicar...

- Escriviu el codi següent i executeu-lo.
- Podeu provar altres formats d'impressió.
 - E.g. què passa si el caràcter
l'imprimeixo com un nombre decimal?

```
#include <stdio.h>

#define MAX 100

int main(){

    printf("Hola!");
    printf("Hola amb salt de linia.\n");


    return 0;
}
```

Per practicar...

- Escriviu el codi següent i executeu-lo.
- Podeu provar altres formats d'impressió.
 - E.g. què passa si el caràcter l'imprimeixo com un nombre decimal?

```
#include <stdio.h>

#define MAX 100

int main(){

    printf("Hola!");
    printf("Hola amb salt de linia.\n");

    // Sortida formatejada
    int a = 34;
    char c = 'A';
    float d = 9.99;

    printf("a es %d\n", a);
    printf("c es %c\n", c);
    printf("d es %f\n", d);
    printf("MAX es %d\n", MAX);

    return 0;
}
```

Per practicar...

- Escriviu el codi següent i executeu-lo.
- Podeu provar altres formats d'impressió.
 - E.g. què passa si el caràcter l'imprimeixo com un nombre decimal?

```
#include <stdio.h>

#define MAX 100

int main(){

    printf("Hola!");
    printf("Hola amb salt de linia.\n");

    // Sortida formatejada
    int a = 34;
    char c = 'A';
    float d = 9.99;

    printf("a es %d\n", a);
    printf("c es %c\n", c);
    printf("d es %f\n", d);
    printf("MAX es %d\n", MAX);
    printf("d es %.2f\n", d);
    printf("d es %.4f\n", d);

    return 0;
}
```


Per practicar...

- Escriviu el codi següent i executeu-lo.
- Podeu provar altres formats d'impressió.
 - E.g. què passa si el caràcter l'imprimeixo com un nombre decimal?

```
#include <stdio.h>

#define MAX 100

int main(){

    printf("Hola!");
    printf("Hola amb salt de linia.\n");

    // Sortida formatejada
    int a = 34;
    char c = 'A';
    float d = 9.99;
    int x = 5, y = 10;

    printf("a es %d\n", a);
    printf("c es %c\n", c);
    printf("d es %f\n", d);
    printf("MAX es %d\n", MAX);
    printf("d es %.2f\n", d);
    printf("d es %.4f\n", d);
    printf("x is %d and y is %d\n", x, y );

    return 0;
}
```

Llegir de teclat

A teoria:

```
var numeret: enter; fvar
...
escriure("Introdueix un numero");
llegir(numeret);
...
```

Crides més freqüents:



```
1 scanf ("%d", &total);    //int
2 scanf ("%f", &radi);     //float
3 scanf (" %c", &inicial); //char
```

No us oblideu el
símbol "&"

Al llegir caràcters,
aquest espai és
necessari

Llegir de teclat

A teoria:

```
var numeret: enter; fvar
...
escriure("Introdueix un numero");
llegir(numeret);
...
```

En C:

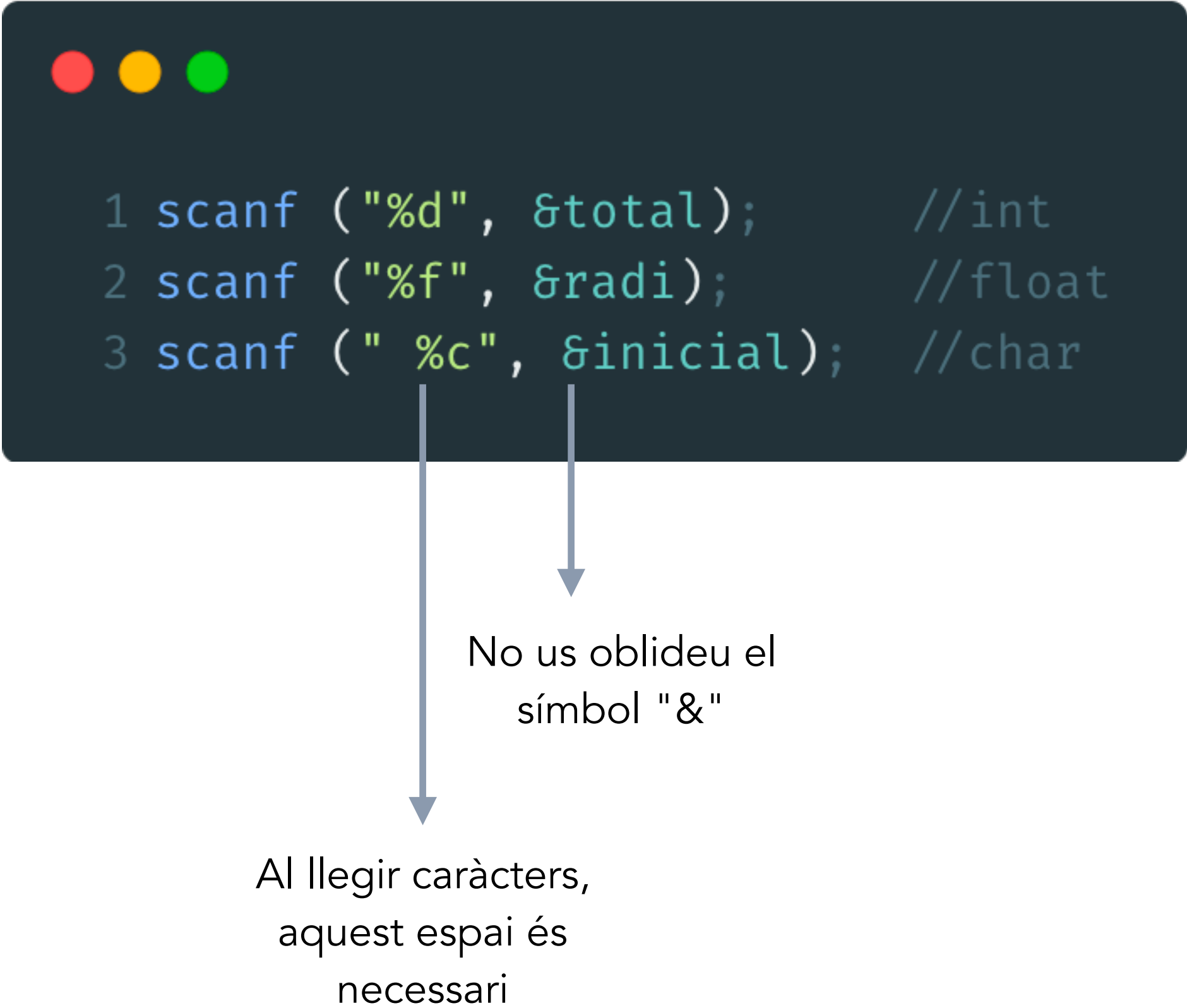
- Farem servir la funció **scanf()**, disponible a la llibreria **<stdio.h>**

```
int scanf(const char *format, ...)
```

```
scanf("%tipus", &variable)
```

- Llegeix entrada formatejada de l'entrada estàndard (stdin, o sigui, el teclat)
- Format: segueix els mateixos formats que printf

Crides més freqüents:



```
1 scanf ("%d", &total); //int
2 scanf ("%f", &radi); //float
3 scanf (" %c", &inicial); //char
```

No us oblideu el
símbol "&"

Al llegir caràcters,
aquest espai és
necessari

Per practicar...

```
#include <stdio.h>

int main(){

    int a;
    char c;
    float d;

    // Llegir un numero
    printf("Introdueix un numero\n");
    scanf("%d", &a);

    // Llegir un caracter
    printf("Introdueix un caracter\n");
    scanf(" %c", &c); // Ojo al truco de l'espai en blanc

    // Llegir un float
    printf("Introdueix un valor decimal\n");
    scanf("%f", &d);

    // Imprimir tot
    printf("L'enter es %d, el caracter es %c i el decimal es %f\n", a, c, d);

    return 0;
}
```

A CASA...

EXERCICIS L1 del MOODLE

EL PROPER DIA...

OPERADORS